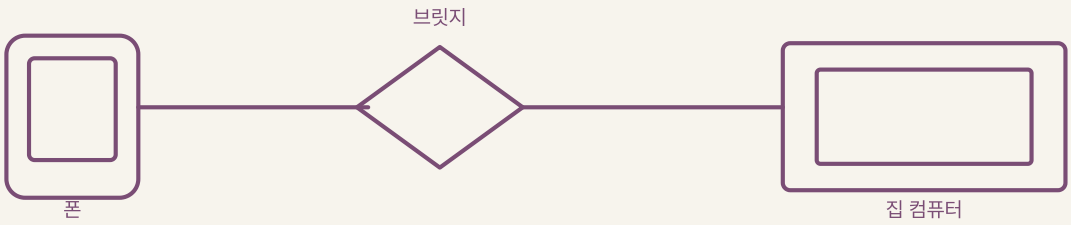
AI WORKSHOP / 03

폰에서 내 컴퓨터 AI 쓰기

Connect your phone to the AI on your PC

텔레그램으로 Claude Code·Codex를 잇는 첫 연결 안내서.
설치 전 적합성부터 양방향 확인, 재시작과 오류 표까지.



강대종 · 마이너스베타스튜디오

2026년 9월 전면 개정판

여는 부분

이용 범위

폰에서 내 컴퓨터 AI 쓰기 — 텔레그램으로 Claude Code·Codex 첫 연결
AI WORKSHOP / 03 · 정가 ₩10,000 · 2026년 9월 전면 개정판

지은이	강대종
펴낸곳	마이너스베타스튜디오
초판	2026년 7월 25일
이 판	2026년 9월 전면 개정판
문의	구매하신 판매 페이지의 문의·메시지 기능

이 책의 저작권은 지은이에게 있습니다.

구매자가 할 수 있는 일. 구매자 본인의 학습과 업무에 이 PDF를 읽고, 인쇄하고, 명령을 복사해 자기 컴퓨터에서 실행할 수 있습니다. 본문에 적힌 공개 오픈소스 브릿지 설치·설정은 해당 프로젝트의 라이선스를 따릅니다.

구매자가 할 수 없는 일. 이 원고·PDF를 무단 복제·재배포·재판매하거나, 필자가 운영하는 봇·계정·컴퓨터를 빌려 쓰는 상품으로 오해하고 요구하는 행위는 허용되지 않습니다.

책에 나오는 봇 이름(`aibridge_demo_2026_bot` 등)과 숫자 `123456789`는 따라하기 **예시**입니다. 독자가 자기 봇을 만듭니다. 이 책은 무한 업데이트, 수익, 특정 요금제 적합성을 약속하지 않습니다.

명령·화면은 공개 오픈소스 `claude-telegram-bridge` / `codex-telegram-bridge`의 현재 README와 배포 패키지 도움말을 기준으로 작성했습니다. 프로그램 버전이 올라가면 화면이 달라질 수 있습니다. 그럴 때는 각 공개 저장소 README를 정본으로 보세요.

이 책으로 만드는 첫 결과

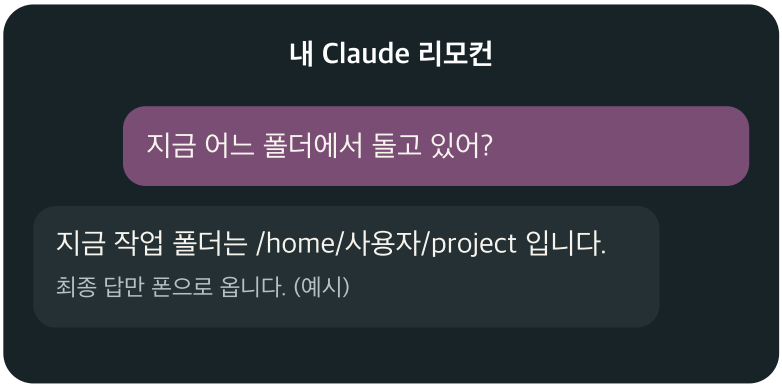
이 책을 순서대로 따라 끝나면, 아래 다섯 가지를 **자기 환경에서** 확인할 수 있어야 합니다.

1. 내 컴퓨터와 운영체제가 이 연결에 맞는지 판단한다.
2. 내 텔레그램 봇 토큰과 chat_id를 화면에 남기지 않고 확보한다.
3. 집 컴퓨터의 Claude Code(또는 Codex)에 브릿지를 설치한다.
4. 폰에서 `/ping`으로 다리가 살아 있는지, 이어서 실제 지시로 AI가 답하는지 **둘 다** 확인한다.

5. 창을 닫거나 컴퓨터를 재시작한 뒤에도 다시 확인하는 명령을 안다.

지하철에서 보내는 문장 예시는 학습용입니다. 매출·절감 시간·성과를 이 책으로 보장하지 않습니다.

조판 예시 화면 — 실제 계정 채팅이 아닙니다



이 책 보는 법

읽는 순서는 설치 이전이 먼저입니다. 프로그램을 깔기 전에 1장을 통과하세요.

1. **적합성** — 내 컴퓨터·AI 도구·운영체제가 이 연결을 할 수 있는지.
2. **권한과 토큰** — 봇 열쇠와 내 chat_id.
3. **연결** — 브릿지 설치와 설정 마법사.
4. **양방향 확인** — `/ping`과 실제 지시의 기댓값이 다른 점.
5. **재시작과 오류** — tmux, 워치독, 실패 표.

`[따라하기]`가 붙은 절은 실습입니다. 회색 상자 명령을 복사해 터미널에 붙여 넣습니다. 각 실습 뒤에는 **기대 결과**와 **다음 행동**을 적었습니다.

캡처 그림은 공개 브릿지 화면을 바탕으로 하되, 토큰·chat_id·개인 경로는 가렸거나 예시 값으로 바꿨습니다. 어두운 말풍선 조판은 동작 흐름을 설명하는 **예시 화면**이며 실제 소프트웨어 화면이 아닙니다. 실제 확인 가능한 출력(`--help`, `/ping` 문구)과 예시를 본문에서 구분해 표시합니다.

막히면 해당 장 끝의 표나 **8장 실패별 원인·확인·다음 행동**으로 가세요. 운영체제 차이는 macOS / Linux / Windows WSL을 과장 없이 적었습니다.

상세 목차

아래 숫자는 이 판매 PDF에서 각 장·절이 실제로 시작하는 쪽입니다.
PDF 뷰어의 북마크에서도 같은 장·절로 바로 이동할 수 있습니다.

- **표지** ⋯ 1
- **여는 부분** ⋯ 2
- 이용 범위 ⋯ 2
- 이 책으로 만드는 첫 결과 ⋯ 2
- 이 책 보는 법 ⋯ 3
- **1장. 설치 전에 맞는지 확인한다** ⋯ 6
- 1.1 이 연결이 하는 일과 하지 않는 일 ⋯ 6
- 1.2 Claude Code와 Codex, 무엇을 연결하나 ⋯ 7
- 1.3 운영체제 경로 — macOS / Linux / Windows WSL ⋯ 7
- 1.4 시작 전 적합성 점검표 ⋯ 7
- 1.5 안전·비용·한계 한눈에 ⋯ 8
- **2장. 권한과 비밀값 — 봇 토큰과 chat_id** ⋯ 9
- 2.1 필요한 것 체크리스트 ⋯ 9
- 2.2 텔레그램 봇이 뭔가요? — 3분 개념 ⋯ 9
- 2.3 [따라하기] BotFather로 내 봇 만들기 ⋯ 9
- 2.4 [따라하기] 내 chat_id 알아내기 ⋯ 10
- 2.5 토큰과 chat_id, 이 둘만 기억하면 된다 ⋯ 11
- **3장. 집 컴퓨터의 AI가 준비됐는지 확인** ⋯ 12
- 3.1 Claude Code가 먼저 깔려 있어야 한다 (전제) ⋯ 12
- 3.2 [따라하기] 설치·로그인 됐는지 확인 ⋯ 12
- 3.3 tmux가 뭐고 왜 꼭 필요한가 ⋯ 13
- 3.4 운영체제별 주의점 (윈도우·맥·리눅스) ⋯ 13
- **4장. Claude 브릿지 설치하고 첫 연결** ⋯ 14
- 4.1 설치 방법 두 가지 — 마법사 vs 수동 ⋯ 14
- 4.2 [따라하기] pipx로 브릿지 설치 ⋯ 14
- 4.3 [따라하기] 설정 마법사 6단계 완주 ⋯ 16
- 4.4 [따라하기] 토큰·chat_id 안전하게 저장 (수동 이해) ⋯ 17

- 4.5 [따라하기] tmux에 Claude 띄우고 훅 등록 (수동 이해) ⋯ 19
- 4.6 [따라하기] 첫 실행 & /ping 으로 확인 ⋯ 20
- 4.7 "됐다!" 첫 진짜 명령 보내보기 ⋯ 21
- 4.8 안 되면? 4장 전용 응급처치 3줄 ⋯ 22
- **5장. 양방향 확인 — 다리가 살아 있는지, AI가 답하는지** ⋯ 23
- 5.1 두 종류의 성공을 섞지 말 것 ⋯ 23
- 5.2 [따라하기] 확인 순서 한 바퀴 ⋯ 23
- 5.3 운영체제별 확인만 다른 점 ⋯ 24
- 5.4 실제 출력과 예시 화면을 구분하는 법 ⋯ 24
- **6장. Codex 브릿지도 연결하기** ⋯ 25
- 6.1 Codex 브릿지는 뭐가 같고 뭐가 다른가 ⋯ 25
- 6.2 [따라하기] codex-telegram-bridge 설치 ⋯ 25
- 6.3 [따라하기] 봇을 새로 팔까, 재활용할까 ⋯ 26
- 6.4 [따라하기] 설정하고 첫 연결 ⋯ 26
- 6.5 Claude 봇과 Codex 봇 안 헷갈리게 ⋯ 27
- **7장. 재시작과 상시 구동 — 안 끊기게 만들기** ⋯ 28
- 7.1 왜 자꾸 끊기나 — 세 가지 원인 ⋯ 28
- 7.2 [따라하기] tmux로 세션 살려두기 ⋯ 28
- 7.3 [따라하기] 워치독으로 자동 복구 ⋯ 29
- 7.4 [따라하기] 컴퓨터 재부팅에도 살아남기 ⋯ 29
- 7.5 잘 돌고 있나 확인하는 습관 (doctor) ⋯ 30
- **8장. 실패별 원인·확인·다음 행동** ⋯ 30
- 8.1 /ping은 오는데 진짜 답이 안 와요 ⋯ 30
- 8.2 봇이 아무 반응이 없어요 ⋯ 30
- 8.3 externally-managed-environment 오류가 떠요 ⋯ 31
- 8.4 윈도우에서 python3가 없대요 ⋯ 31
- 8.5 답이 두 번 와요 / 메시지가 꼬여요 ⋯ 31
- 8.6 그 외 자주 막히는 지점 ⋯ 31
- **9장. 학습용 가상 사례 — 첫 연결부터 복구까지** ⋯ 32
- 9.1 사례 한 줄 요약 ⋯ 32
- 9.2 [따라하기] 민서의 당일 순서 ⋯ 32

- 9.3 Windows WSL 경로만 다른 가상 사례 … 32
- 9.4 연결한 뒤에 쓸 수 있는 기능 10가지 … 32
- 9.5 사례가 끝난 뒤 독자가 남기는 기록 … 33
- **부록 A. 보안·토큰 관리 상식** … 34
- A.1 토큰이 노출되면 벌어지는 일 … 34
- A.2 [따라하기] 토큰 안전 보관·교체 … 34
- A.3 --dangerously-skip-permissions, 켤까 말까 … 34
- **부록 B. 명령·환경변수 치트시트** … 35
- B.1 자주 쓰는 명령 … 35
- B.2 Codex 설정 파일과 핵심 환경변수 … 35
- **닫는 글 — 이용 범위와 다음 행동** … 37

1장. 설치 전에 맞는지 확인한다

이 장은 패키지를 설치하기 전에 읽습니다. 여기서 멈추는 것이 맞는 경우도 있습니다.

1.1 이 연결이 하는 일과 하지 않는 일

집에 **컴퓨터에서 일하는 AI 조수**(Claude Code 또는 Codex)가 이미 있습니다. 문제는 그 조수에게 일을 시키려면 **그 컴퓨터 앞**에 있어야 한다는 점입니다.

이 책이 놓는 브릿지(다리)는 조수에게 전화선 하나를 연결합니다. 그 전화가 **내가 만든 텔레그램 봇**입니다. 밖에서 폰으로 봇에게 말하면, 그 말이 집 컴퓨터의 AI에게 전달되고, 최종 답만 다시 폰으로 돌아옵니다.

한 줄: **폰(텔레그램) → 다리(브릿지) → 집 컴퓨터의 AI → 다시 폰.**

하지 않는 일:

- 클라우드에 새 AI를 빌려 주는 서비스가 아닙니다.
- 필자의 봇이나 컴퓨터를 원격으로 쓰는 상품이 아닙니다.
- Claude Code·Codex 자체의 설치·로그인·구독을 대신해 주지 않습니다.
- 컴퓨터가 꺼져 있거나 인터넷이 끊기면 동작하지 않습니다.

1.2 Claude Code와 Codex, 무엇을 연결하나

둘 다 “컴퓨터에서 일하는 AI 조수”입니다.

- **Claude Code** — Anthropic이 만든 조수. 4장에서 연결합니다.
- **Codex** — OpenAI가 만든 조수. 6장에서 연결합니다.

하나만 써도 됩니다. 해당 장만 따라 하세요. 브릿지는 **서로 다른 프로그램**이라 각각 설치합니다. 하나 깔았다고 다른 쪽이 같이 깔리지 않습니다.

1.3 운영체제 경로 — macOS / Linux / Windows WSL

이 책의 기본 실습 경로는 **macOS 또는 Linux 터미널에서 tmux를 쓰는 것**입니다. Windows는 **WSL 안의 Linux**를 기준으로 안내합니다. 순수 Windows 실험 모드(`host`)는 도움말에 있으나, 첫 연결은 WSL을 권합니다.

운영체제	이 책이 안내하는 경로	과장하지 않는 한계
macOS	기본 경로. tmux + 사용자 LaunchAgent	덮개 닫힘·절전 시 연결이 멈출 수 있음
Linux	기본 경로. tmux + 사용자 systemd	배포판마다 패키지 이름이 다를 수 있음
Windows	**WSL Linux** 안에서 mac/Linux와 같은 명령	일반 PowerShell만으로는 이 책의 tmux 경로가 그대로 되지 않음
Windows PowerShell	`py` 로 pipx를 까는 보조 안내	첫 연결의 정본이 아님

Windows에서 `python3`가 없고 `py`만 있는 것은 고장이 아닙니다. 4.2와 8장을 보세요.

1.4 시작 전 적합성 점검표

아래를 **모두** 만족할 때만 2장으로 갑니다.

항목	통과 기준	아직이면
AI 도구	컴퓨터에 Claude Code 또는 Codex가 **이미** 설치·로그인됨	각 도구 공식 안내로 먼저 설치. 이 책은 그 단계를 다루지 않음
전원·네트워크	집 컴퓨터가 켜져 있고 인터넷에 연결됨	꺼진 컴퓨터로는 폰 연결이 되지 않음
터미널	검은 창에 글자를 붙여 넣고 Enter를 누를 수 있음	붙여넣기부터 연습
텔레그램	폰(또는 컴퓨터)에서 텔레그램에 로그인됨	앱 설치·로그인
시간	첫 연결에 한 번에 끝날 시간을 비움	토큰을 채팅에 남긴 채 중단하지 말 것

통과하지 못하면 설치를 시작하지 마세요. 특히 AI 도구가 없는 상태에서 브릿지부터 깔면, `/ping`은 와도 실제 답이 없는 상태에 빠지기 쉽습니다.

1.5 안전·비용·한계 한눈에

- **보안.** 봇 토큰은 비밀번호와 같습니다. 새면 남이 봇을 조종할 수 있습니다(부록 A).
- **한 사람만.** 이 브릿지는 **당신의 텔레그램 채팅 하나**에만 답합니다. 남이 봇 주소를 알아도 반응하지 않게 설정합니다.
- **비용·약관.** 24시간 다리를 당신 계정으로 운영하는 것이 각 서비스 요금제·약관에 맞는지는 **본인이 판단**합니다. 이 책은 특정 요금제 적합성을 보장하지 않습니다.
- **권한 건너뛰기.** Claude를 `--dangerously-skip-permissions`로 켜면, 파일 변경·명령 실행 확인을 묻지 않습니다. 뜻을 이해하고 켜세요(부록 A.3).
- **한계.** 컴퓨터 전원·절전·인터넷이 끊기면 워치독도 소용이 없습니다. 안 끊기게 다지는 법은 7장입니다.

2장. 권한과 비밀값 — 봇 토큰과 chat_id

2.1 필요한 것 체크리스트

구분	준비물	확인
컴퓨터	Claude Code(또는 Codex) 설치·로그인 완료	3장에서 확인
컴퓨터	Python 3, tmux	3장에서 확인
폰	텔레그램 앱	설치·로그인
비밀값	봇 토큰, 내 chat_id	이 장에서 발급

2.2 텔레그램 봇이 뭔가요? — 3분 개념

봇(bot) 은 사람 대신 자동으로 움직이는 텔레그램 계정입니다. 우리는 "내 전용 봇"을 하나 만들어, 그 봇을 **집 컴퓨터와 폰 사이의 우체통**으로 씁니다. 폰에서 봇에게 보낸 말은 컴퓨터가 가져가고, 컴퓨터의 답은 봇을 통해 폰으로 돌아옵니다.

봇을 만들어 주는 것은 텔레그램 공식 봇인 **BotFather**입니다.

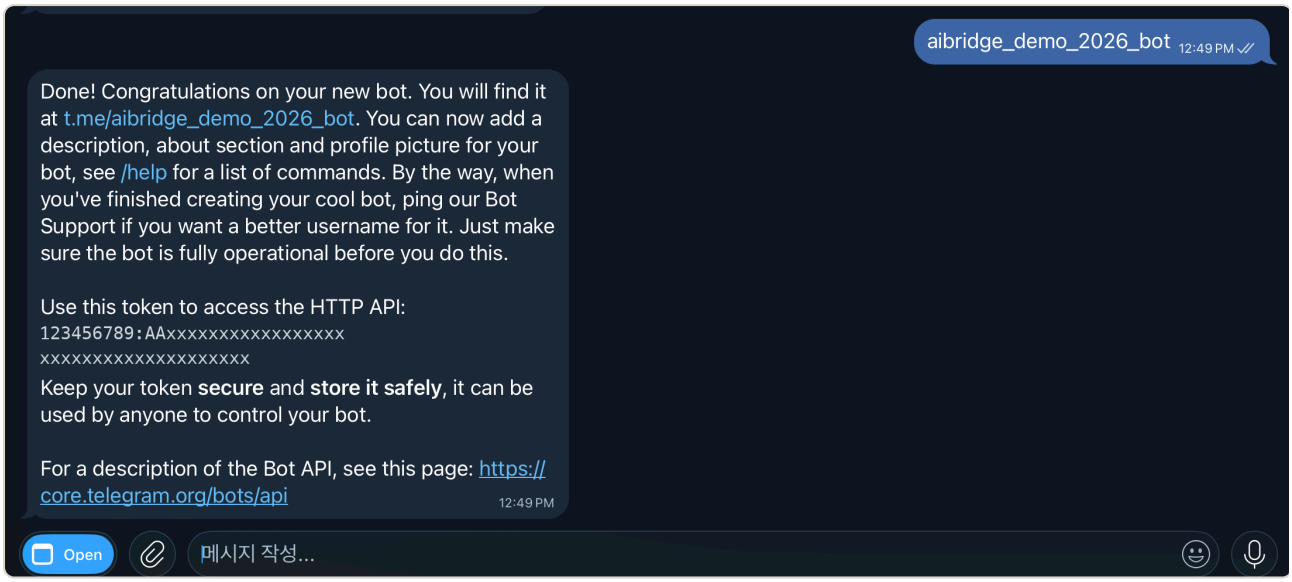
권한의 핵심은 두 가지입니다.

- **토큰** — 봇을 조종하는 열쇠. 브릿지 프로그램만 읽게 합니다.
- **chat_id** — "이 채팅의 나"를 가리키는 숫자. 다른 사람은 같은 봇에게 말해도 답이 오지 않게 합니다.

2.3 [따라하기] BotFather로 내 봇 만들기

1. 텔레그램에서 `@BotFather` 를 검색해 엽니다(파란 체크가 붙은 공식 계정).
2. `/newbot` 을 보냅니다.
3. 봇의 **표시 이름**을 묻습니다. 예: `AI Bridge Demo`.
4. 봇의 **아이디(username)** 를 묻습니다. 세상에 하나뿐이어야 하고 **`bot` 으로 끝나야** 합니다. 예: `aibridge_demo_2026_bot`.
5. BotFather가 **토큰**을 줍니다. `1234567890:AbCd...` 모양(숫자, 콜론, 긴 글자)입니다. **복사**하세요.

토큰은 **비밀**입니다. 컴퓨터의 로컬 파일에만 붙여넣고, 텔레그램 채팅이나 공개 저장소에는 넣지 마세요.



기대 결과

BotFather가 토큰 한 줄을 줍니다. 화면에 토큰이 보입니다. 이 값을 채팅에 다시 붙여 넣지 말고, 다음 단계까지 클립보드 또는 로컬 메모에만 둡니다.

다음 행동: 방금 만든 봇과의 채팅을 열고 아무 말이나 보낸 뒤, 2.4로 갑니다. 가능하면 chat_id는 4장의 `setup` 마법사에 맡깁니다.

2.4 [따라하기] 내 chat_id 알아내기

브릿지는 **딱 한 채팅**(당신)에게만 답합니다. 그 "당신"이 누군지 알려 주는 숫자가 **chat_id**입니다.

1. 방금 만든 봇과의 채팅을 열고 아무 말이나 보냅니다(예: `hello`). 봇은 먼저 말을 걸 수 없어서, **당신이 먼저** 말해야 합니다.
2. 가능하면 이 단계는 건너뛰고 4장의 `setup` 마법사에 chat_id를 맡기세요.
   수동으로 볼 때는 토큰을 명령에 직접 적지 마세요(터미널 기록에 남습니다).

```
python3 - <<'PY'
from getpass import getpass
import json, urllib.request

token = getpass("Bot token: ").strip()
with urllib.request.urlopen(
    f"https://api.telegram.org/bot{token}/getUpdates", timeout=20
) as response:
    updates = json.load(response).get("result", [])
for item in updates:
    chat = item.get("message", {}).get("chat", {})
    if "id" in chat:
        print("chat_id:", chat["id"])
PY
```

3. 응답에서 `"chat":{"id":123456789,...}` 의 **숫자**가 chat_id입니다. 위 Python은 숫자만 출력합니다. 응답이 비어 있으면, 봇에게 메시지를 한 번 더 보내고 다시 실행하세요.



그림의 명령 뒤에 붙은 `python3 -m json.tool | grep ...` 은 **응답을 보기 좋게 줄이려고** 붙인 것이고, `sed ...` 는 **필자의 실제 chat_id를 `123456789` 로 가리려고** 붙인 것입니다. 여러분은 위 Python 명령만 실행하면 됩니다. 토큰은 화면·명령 이력에 나타나지 않으며 결과에는 chat_id 숫자만 나옵니다.

> 기대 결과
>
> `chat_id:` 뒤에 정수 하나가 출력됩니다. 토큰 문자열은 출력되지 않아야 합니다.
>
> **다음 행동:** 숫자만 기억하거나 로컬에 적어둡니다. 2.5를 읽고 3장으로 갑니다.

2.5 토큰과 chat_id, 이 둘만 기억하면 된다

앞으로 계속 쓰는 두 값입니다.

- **봇 토큰** = 봇의 열쇠(비밀). `1234567890:AbCd...`

- **chat_id** = "나"를 가리키는 번호. `123456789`

이 둘을 컴퓨터에 안전하게 저장하는 것은 4장(Claude)·6장(Codex)에서 안내합니다. 토큰을 채팅에 보내 버렸다면 즉시 부록 A.2에서 재발급하세요.

3장. 집 컴퓨터의 AI가 준비됐는지 확인

3.1 Claude Code가 먼저 깔려 있어야 한다 (전제)

이 책은 **Claude Code(또는 Codex)가 이미 설치·로그인된 상태**에서 시작합니다. 아직 안 깔았다면 각 도구의 공식 안내로 먼저 설치·로그인하세요. 여기서는 "깔려 있는지 확인하는 법"만 봅니다.

3.2 [따라하기] 설치·로그인 됐는지 확인

컴퓨터 터미널에 아래를 쳐 보세요.

```
claude --version
```

버전 숫자가 나오면 설치돼 있는 것입니다. `command not found` 가 나오면 아직 설치 전입니다.
그리고:

```
tmux -V
python3 --version
```

두 개 모두 버전이 나오면 준비 끝입니다. (예: `tmux 3.4`, `Python 3.12.3`)

우분투/WSL은 대부분 `python3` · `tmux` 가 이미 있거나 `sudo apt install tmux` 로 금방 깔립니다.

Codex만 연결할 계획이면 같은 방식으로 `codex --version` 을 확인하세요.

> 기대 결과
>
> 세 명령이 각각 버전 한 줄을 출력합니다. 오류 문구가 없어야 합니다.
>
> **다음 행동:** 하나라도 실패하면 해당 도구를 먼저 고칩니다. 모두 통과하면 3.3을 읽고 4장으로 갑니다.

3.3 tmux가 뭐고 왜 꼭 필요한가

tmux 는 “터미널 창을 계속 켜 두는 상자”입니다. 브릿지가 폰의 메시지를 넣어 줄 **Claude Code 화면이 항상 켜져 있어야** 하는데, 보통 터미널 창은 닫으면 그 안의 프로그램도 같이 꺼집니다. tmux 안에서 돌리면 **창을 닫아도 안 꺼집니다.** 그래서 밖에서 폰으로 부릴 수 있습니다. 자세한 활용은 7장입니다.

3.4 운영체제별 주의점 (윈도우·맥·리눅스)

- **맥 / 리눅스:** 그대로 tmux를 씁니다. 이 책의 기본 경로입니다.
- **윈도우:** 기본은 **WSL**(윈도우 안의 리눅스) 안에서 tmux로 돌립니다. 순수 윈도우용 실험 모드도 있지만, 처음이라면 WSL 경로를 권합니다.
- 어느 쪽이든 **Python 3**가 필요합니다. 윈도우 파워셸에서는 `python3` 대신 `py` 를 쓰는 경우가 많습니다(4장에서 안내).

4장. Claude 브릿지 설치하고 첫 연결

이 장 하나만 끝내면, **폰에서 보낸 메시지가 집 컴퓨터의 Claude Code로 들어가고, Claude가 낸 답이 다시 폰으로 돌아옵니다.** 이 책의 심장입니다. 천천히, 한 줄씩 따라오세요. 중간에 막히면 이 장 맨 끝 **4.8 응급처치**로 바로 가면 됩니다.

준비물 확인(2·3장에서 챙긴 것):

- 컴퓨터에 **Claude Code**가 깔려 있고 로그인돼 있다
- 텔레그램에서 만든 **봇 토큰** (`1234567890:AbCd...` 모양)
- 내 **chat_id** (`123456789` 같은 숫자)
- 컴퓨터에 **Python 3**와 **tmux**가 있다 (없으면 3장 참고)

4.1 설치 방법 두 가지 — 마법사 vs 수동

브릿지를 까는 길은 두 갈래입니다.

방법	이런 분께	특징
① 설정 마법사	처음이라 빠르게 되게 하고 싶다	명령 한 줄이 6단계를 알아서 진행
② 수동 설치	각 단계가 뭘 하는지 알고 싶다	파일을 직접 만들어 원리를 이해

이 장은 **① 마법사(4.2~4.3)를 기본 길**로 안내하고, 마법사가 만들어 주는 것들을 **수동(4.4~4.5)으로도** 짚습니다. 마법사가 잘 됐다면 4.4~4.5는 "이런 게 만들어졌구나" 확인용으로 읽으면 됩니다.

마법사는 편하지만, 나중에 문제가 생기면 "어떤 파일이 뭘 하는지" 알아야 고칠 수 있습니다. 그래서 원리도 같이 담았습니다.

4.2 [따라하기] pipx로 브릿지 설치

`pipx`는 "파이썬으로 만든 프로그램을 깔끔하게 설치해 주는 도구"입니다. 먼저 브릿지를 깝니다.

```
pipx install claude-telegram-bridge
```

설치가 끝나면 `claude-telegram-bridge`라는 명령을 쓸 수 있게 됩니다.

`pipx: command not found` 가 뜨면

`pipx`가 아직 없다는 뜻입니다. 딱 한 번만 깔고 터미널을 새로 열면 됩니다.

```
python3 -m pip install --user pipx
python3 -m pipx ensurepath
```

터미널을 **닫았다 다시 열고**, 위의 `pipx install ...`을 한 번 더 실행하세요.

윈도우 파워셸에는 보통 `python3`가 없고 `py`를 씁니다.

```
py -m pip install --user pipx
py -m pipx ensurepath
```

우분투 / WSL 사용자: `pip install`을 직접 하면 `error: externally-managed-environment`가 뜰 수 있습니다. 이건 최신 우분투의 안전장치입니다. 위처럼 **pipx를 쓰는 걸 권장**합니다. (자세한 건 8.3 참고)

설치 확인 — 아래를 치면 쓸 수 있는 기능 목록이 나옵니다.

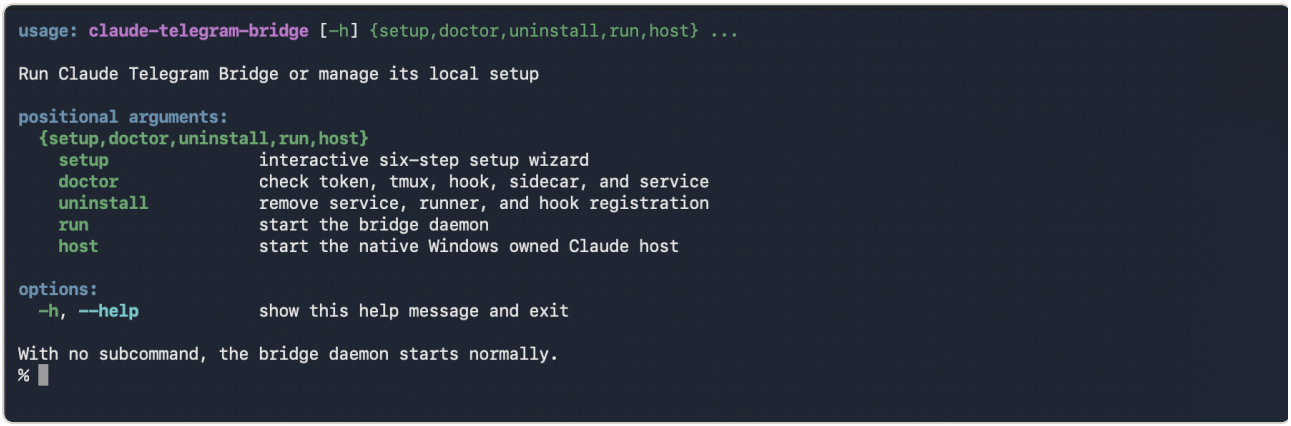
```
claude-telegram-bridge --help
```

이런 화면이 나오면 성공입니다. 아래는 격리 설치한 배포 패키지에서 실제로 확인한 도움말입니다.

```
usage: claude-telegram-bridge [-h] {setup,doctor,uninstall,run,host} ...

Run Claude Telegram Bridge or manage its local setup

  setup       interactive six-step setup wizard
  doctor      check token, tmux, hook, sidecar, and service
  uninstall   remove service, runner, and hook registration
  run         start the bridge daemon
  host        start the native Windows owned Claude host
```



기대 결과

`setup`, `doctor`, `uninstall`, `run`, `host`가 보입니다.

다음 행동: 4.3 마법사를 실행합니다. `command not found`이면 터미널을 새로 연 뒤 4.2를 다시 합니다.

4.3 [따라하기] 설정 마법사 6단계 완주

가장 쉬운 길입니다. 아래 한 줄이면 마법사가 시작됩니다.

```
claude-telegram-bridge setup
```

마법사는 이 6가지를 순서대로 해 줍니다.

1. **봇 토큰 확인** — 아까 받은 토큰이 진짜 살아 있는지 점검
2. **chat_id 자동 감지** — 봇에게 `/start`를 보내면 마법사가 당신의 chat_id를 알아챕니다
3. **비밀 파일 2개 생성** — `token.json`, `token-registry.json` (다음 절 설명)
4. **SessionStart 훅 설치** — Claude가 낸 답을 브릿지가 찾을 수 있게 연결
5. **로컬 서비스/워치독 설치** — 꺼지면 다시 살아나게 (7장에서 자세히)
6. **테스트 메시지 1통 발송** — 폰으로 "설치 완료" 메시지가 오면 끝

```
% claude-telegram-bridge setup
Claude Telegram Bridge setup
    You need a BotFather token, Telegram /start, and Claude Code. tmux is also required.

[1/6] Paste and validate the BotFather token
[ok] token valid: @aibridge_demo_2026_bot

[2/6] Connect your Telegram chat
_   Open @aibridge_demo_2026_bot, send /start, and wait up to 180s.
[ok] chat id detected: 123456789 (You)

[3/6] Write token.json, token-registry.json, and private config
[ok] wrote private config: /Users/Shared/clbdemo/.config/claude-telegram-bridge/bridge.env
[ok] wrote token files: /Users/Shared/clbdemo/.config/claude-telegram-bridge/token.json, /Users/Shared/clbdemo/.config/claude-telegram
-bridge/token-registry.json

[4/6] Install and register the Claude SessionStart hook
[ok] installed SessionStart hook: /Users/Shared/clbdemo/.config/claude-telegram-bridge/claude-telegram-bridge-session-start.sh
[ok] Claude settings SessionStart hook registered

[5/6] Install the bridge service and watchdog
[ok] installed runner: /Users/Shared/clbdemo/.local/bin/claude-telegram-bridge-run
[warn] service/watchdog install skipped

[6/6] Send a setup-complete test message
_   Test message skipped by option.
```

그림의 `chat id detected: 123456789 (You)`에서 숫자와 이름은 **필자 것을 가린 자리표시자**입니다. 여러분 화면에는 진짜 숫자와 본인 이름이 나옵니다. 또 이 그림은 책 만들면서 찍은 것이라 **5·6단계를 일부러 건너뛴 상태**(`skipped`)입니다 — 필자 컴퓨터에 상주 서비스를 남기지

않으려고 그랬습니다. 여러분이 그냥 `setup` 을 돌리면 그 두 줄도 `[ok]` 로 채워지고, 마지막에 폰으로 "설치 완료" 메시지가 옵니다.

마법사가 중간에 **"Start the Claude tmux session now? [Y/n]"** 이라고 물으면, 그냥 **Enter(=예)** 를 누르면 됩니다. 그러면 Claude가 돌아갈 방(tmux 세션)까지 만들어 줍니다.

마법사가 끝에 폰으로 테스트 메시지를 보냈다면, 4.4~4.5는 **읽고 넘어가도** 됩니다(마법사가 이미 다 해 둠). 원리가 궁금하면 계속 읽으세요. 곧장 확인하려면 **4.6**으로.

> 기대 결과
>
> 폰으로 설치 완료 메시지가 오거나, 마법사 마지막에 실패 항목이 0으로 끝납니다. 캡처의 `skipped` 는 필자 촬영 상태이며 독자 기본 경로가 아닙니다.
>
> **다음 행동:** 4.6에서 `/ping` 을 보냅니다. 마법사를 건너뛴 경우에만 4.4~4.5를 실행합니다.

4.4 [따라하기] 토큰·chat_id 안전하게 저장 (수동 이해)

마법사가 만들어 준(또는 수동으로 만들) **비밀 파일 2개**를 이해할 차례입니다.

- `token.json` — 봇 토큰을 담는 파일
- `token-registry.json` — "이 토큰의 주인은 나"라고 표시하는 장부

토큰은 **절대 화면·채팅·인터넷에 남으면 안 되는 비밀번호**입니다. 그래서 화면에 안 보이게 붙여넣는 방법을 씁니다.

```
python3 - <<'PY'
from getpass import getpass
import hashlib, json, pathlib

token = getpass("봇 토큰을 붙여넣고 Enter: ").strip()
root = pathlib.Path.home() / ".config/claude-telegram-bridge"
root.mkdir(parents=True, exist_ok=True)
(root / "token.json").write_text(json.dumps({"token": token}))
(root / "token.json").chmod(0o600)
token_id = hashlib.sha256(token.encode()).hexdigest()[:16]
(root / "token-registry.json").write_text(json.dumps({
    "tokens": {"default": {"token_id": token_id, "mode": "polling",
        "owner": "claude-telegram-bridge", "expected_consumer": "claude",
        "allow_delete_webhook": False}}}, indent=2))
(root / "token-registry.json").chmod(0o600)
print("비밀 파일 2개를 만들었습니다")
PY
```

- Python의 `getpass`가 입력을 화면에 표시하지 않으므로 macOS 기본 zsh·Bash·리눅스에서 같은 명령을 쓸 수 있습니다.
- `chmod(0o600)`은 "이 파일은 나만 읽는다"는 잠금입니다.

봇 토큰이 새어나가면 남이 당신 봇을 조종할 수 있습니다. 토큰은 은행 비밀번호처럼 다루세요. (부록 A에서 더 자세히)

```
% cd ~/.config/claude-telegram-bridge
% ls -l
total 16
-rw-r--r--@ 1 user  wheel  221 Jul 25 12:56 token-registry.json
-rw-------@ 1 user  wheel   59 Jul 25 12:56 token.json
% cat token-registry.json
{
  "tokens": {
    "default": {
      "token_id": "e026a91eb03b22e4",
      "mode": "polling",
      "owner": "claude-telegram-bridge",
      "expected_consumer": "claude",
      "allow_delete_webhook": false
    }
  }
}
%
```

기대 결과

터미널에 `비밀 파일 2개를 만들었습니다` 만 보입니다. 토큰 원문은 출력되지 않습니다.

다음 행동: 4.5에서 tmux와 훅을 등록합니다.

4.5 [따라하기] tmux에 Claude 띄우고 훅 등록 (수동 이해)

브릿지는 **계속 켜져 있는 Claude Code 화면**에 메시지를 넣어 줍니다. 그 "꺼지지 않는 화면"이 바로 **tmux 세션**입니다.

① Claude를 tmux 안에서 시작

```
tmux -L default new -s claude
claude --dangerously-skip-permissions
```

`--dangerously-skip-permissions` 는 평소 Claude Code가 파일을 고치거나 명령을 실행하기 전에 "해도 돼요?" 하고 묻는 절차를 건너뛰게 합니다. 밖에서 폰으로 부릴 때는 그 물음에 답할 사람이 터미널 앞에 없습니다. **편한 대신, Claude가 물어보지 않고 당신 컴퓨터에서 작업합니다.** 이 뜻을 이해하고 켜세요. 부담되면 그냥 `claude` 로 시작해도 브릿지는 됩니다(단, 확인 창이 뜨면 폰 응답이 그동안 멈춥니다). — 부록 A 참고

② 답을 찾을 수 있게 훅 등록

브릿지가 "Claude가 방금 낸 답"을 정확히 집어오려면, `SessionStart` 훅 하나가 필요합니다. 마법사를 썼다면 이미 등록돼 있습니다. 수동 설치를 이해하려면 공개 저장소를 받은 뒤 훅 파일을 짧은 로컬 이름으로 복사합니다. 각 줄은 PDF에서 그대로 복사할 수 있게 경로를 변수로 나눴습니다.

```
REPO="$HOME/claude-telegram-bridge"
HOOK="$HOME/.config/claude-telegram-bridge/session-start.sh"
git clone https://github.com/ssamssae/claude-telegram-bridge.git "$REPO"
mkdir -p "$(dirname "$HOOK")"
cp "$REPO/hooks/claude-telegram-bridge-session-start.sh" \
  "$HOOK"
chmod +x "$HOOK"
```

`~/.claude/settings.json` 에 아래 훅을 등록합니다. 파일이 이미 있으면 덮어쓰지 말고 기존 `hooks` 객체 안에 `SessionStart` 항목을 합치세요.

```
{
  "hooks": {
    "SessionStart": [
      { "hooks": [
        { "type": "command",
          "command": "/ABSOLUTE/PATH/session-start.sh" }
      ] }
    ]
  }
}
```

`/ABSOLUTE/PATH/session-start.sh`에는 같은 터미널에서 `echo "$HOOK"`로 확인한 전체 경로를 넣습니다. `settings.json` 안에서는 `~`가 풀리지 않습니다.

등록했으면 Claude 세션에서 `/clear`를 한 번 실행(또는 새 세션 시작)해 훅이 최소 한 번 켜지게 합니다.



등록이 제대로 됐는지는 위 그림처럼 `claude-telegram-bridge doctor`의 `hook` 두 줄이 `[ok]`인지로 확인하면 가장 확실합니다.

4.6 [따라하기] 첫 실행 & /ping 으로 확인

이제 브릿지를 켭니다. 마법사를 썼다면 이미 돌고 있을 수 있습니다(그럼 바로 아래 `/ping`으로).
수동이라면 필요한 값 3개만 알려주고 실행합니다.

```
CONF="$HOME/.config/claude-telegram-bridge"
export CLB_TOKEN_FILE="$CONF/token.json"
export CLB_TOKEN_REGISTRY="$CONF/token-registry.json"
export CLB_CHAT_ID="123456789"
claude-telegram-bridge
```

폰에서 확인 — 텔레그램에서 내 봇에게 `/ping` 을 보냅니다. 브릿지가 곧바로 답하면 → **토큰·chat_id가 맞다는 뜻**(절반 성공).



위 그림은 컴퓨터 텔레그램에서 찍었지만 폰에서도 **똑같이** 보입니다. 답이 오면 토큰과 chat_id가 맞다는 뜻입니다. `/ping` 성공은 **다리가 살아 있다**는 뜻이지, AI가 일을 끝냈다는 뜻이 아닙니다. 그 구분은 5장입니다.

4.7 "됐다!" 첫 진짜 명령 보내보기

이제 진짜 Claude에게 시켜봅니다. 폰에서 봇에게:

```
지금 어느 폴더에서 돌고 있어?
```

잠시 뒤, 컴퓨터의 tmux Claude 화면에 이 메시지가 뜨고 → Claude가 답을 만들고 → 그 **최종 답만** 폰으로 돌아옵니다. (중간 과정은 안 오고 깔끔한 답만 옵니다.)

폰과 컴퓨터가 이어졌습니다. 이제 밖에서도 이 봇에게 말하면 집 컴퓨터의 Claude가 일합니다.

사진을 보내면 Claude가 그 이미지를 보고 답할 수도 있고, 슬래시 명령 `/context` 를 보내면 현재 상태를 그림으로 받아볼 수도 있습니다. — 9장 실전편에서.

기대 결과

폰에 작업 폴더 경로가 포함된 최종 답이 옵니다. tmux의 Claude 화면에도 같은 질문이 보여야 합니다.

다음 행동: 5장에서 /ping과 실제 답의 차이를 표로 고정합니다. Codex도 연결하려면 6장, 끊김을 줄이려면 7장입니다.

4.8 안 되면? 4장 전용 응급처치 3줄

증상	가장 흔한 원인	확인	다음 행동
`/ping` 은 오는데 **진짜 답이 안 옴**	SessionStart 훅 미등록 또는 등록 후 세션 재시작 안 함	`doctor` 의 hook 줄	4.5 훅 등록 확인 → Claude 세션에서 `/clear` 한 번
봇이 **아무 반응 없음**	토큰/chat_id 틀림, 또는 인터넷	`claude-telegram-bridge doctor`	토큰 재입력, 봇에게 먼저 말 걸기, 네트워크 확인
설치 자체가 **오류**	pipx 미설치 / 우분투 pip 잠금	오류 문구에 `externally-managed` 또는 `command not found`	4.2의 pipx 설치 안내 다시

한 방에 점검하고 싶으면:

```
claude-telegram-bridge doctor
```

토큰·tmux·훅·서비스 상태를 한꺼번에 확인해 어디가 막혔는지 알려 줍니다. 그래도 안 되면 → **8장**에 증상별로 더 있습니다.

5장. 양방향 확인 — 다리가 살아 있는지, AI가 답하는지

4.6과 4.7을 한 번 더, **무엇을 성공으로 볼지**만 고정합니다. 설치를 반복하지 않습니다.

5.1 두 종류의 성공을 섞지 말 것

확인	보내는 것	성공의 뜻	실패 시 먼저 볼 곳
다리 생존	`/ping`	토큰·chat_id·브릿지 프로세스가 맞음	8.2, `doctor`
AI 응답	`지금 어느 폴더에서 돌고 있어?`	tmux 안의 Claude/Codex가 받아 최종 답을 냄	8.1, 훅, `/clear`
재시작 생존	창을 닫거나 재부팅 후 `/ping`	tmux·서비스·워치독	7장

`/ping`만 되고 실제 답이 없으면 **연결의 절반만** 된 것입니다. 이 책을 여기서 끝내지 마세요.

5.2 [따라하기] 확인 순서 한 바퀴

컴퓨터에서:

```
claude-telegram-bridge doctor
```

폰에서 순서대로:

1. `/ping` — 곧바로 짧은 생존 답이 와야 합니다.
2. `지금 어느 폴더에서 돌고 있어?` — 최종 답이 와야 합니다.
3. (선택) 같은 질문을 한 번 더 보내 중복 답 여부를 봅니다. 두 번 오면 8.5입니다.



그림 맨 윗줄의 `| sed ...` 는 필자의 chat_id를 가리려고 붙인 것이니 따라 치지 않아도 됩니다. `[warn]` 은 **고장이 아니라 안내**입니다. 위 그림은 서비스·워치독을 일부러 설치하지 않은 상태라 경고가 3줄 떴고, 맨 아래 `0 failure(s)` 가 "막힌 곳은 없다"는 뜻입니다.

> 기대 결과
>
> `doctor` 에 failure 0. 폰에서 /ping과 실제 지시가 모두 답이 옴. tmux 세션에 질문이 보임.
>
> **다음 행동:** 통과하면 7장으로 상시 구동을 다집니다. Codex를 쓰려면 그전에 6장입니다.

5.3 운영체제별 확인만 다른 점

확인 명령 자체는 같습니다. 다른 것은 **어디에 붙어 보느냐**입니다.

운영체제	세션 다시 보기	비고
macOS / Linux	`tmux -L default attach -t claude`	기본
Windows WSL	WSL 터미널에서 위와 동일	첫 연결 정본
Windows에서 WSL 세션 보기	`WSL.exe -- tmux -L default attach -t claude`	파워셸 보조

빠져나올 때는 `Ctrl+B` 를 누른 뒤 `D` 입니다. 세션은 계속 살아 있습니다.

5.4 실제 출력과 예시 화면을 구분하는 법

- **실제 확인 가능:** `--help` 문구, `doctor` 의 `[ok]` / `[warn]` / `failure` , `/ping` 의 짧은 생존 답, 훅 파일 존재.
- **실제 캡처(가림값):** BotFather·마법사·파일 권한 그림. 숫자와 이름은 예시로 바꿨습니다.
- **조판 예시:** 어두운 말풍선. 동작 순서를 보여 줄 뿐 소프트웨어 화면이 아닙니다.

내 화면의 봇 이름·경로·chat_id가 그림과 다른 것은 정상입니다.

6장. Codex 브릿지도 연결하기

4장을 해냈다면 이 장은 훨씬 쉽습니다. 흐름이 거의 똑같고, **다른 점만** 짚습니다. Claude만 쓰면 이 장을 건너뛰고 7장으로 가도 됩니다.

6.1 Codex 브릿지는 뭐가 같고 뭐가 다른가

같은 것: BotFather로 봇 만들기, chat_id 찾기, "계속 켜진 세션(tmux)에 폰 메시지를 넣어 주는" 원리 — 4장과 동일합니다.

다른 것:

항목	Claude 브릿지	Codex 브릿지
조종 대상	Claude Code	Codex CLI
설치 패키지	`claude-telegram-bridge`	`codex-telegram-bridge`
설정 위치·변수	`~/.config/claude-telegram-bridge/bridge.env` 의 `CLB_*`	`~/.config/telegram-agent-bridge.env` 의 공통 `TAB_*` + REPL 전용 `CRB_*`
기본 모드	보이는 tmux 세션	보이는 tmux 세션(`repl`) — 윈도우는 글자만 쓰는 `exec` 모드도 있음

둘은 **완전히 별개 프로그램**입니다. Codex를 쓰려면 이 장을 따라 **따로** 설치합니다.

6.2 [따라하기] codex-telegram-bridge 설치

먼저 Codex CLI가 깔려 로그인돼 있어야 합니다. 확인:

```
codex --version
```

버전이 나오면 브릿지를 깝니다(4장과 같은 pipx 방식):

```
pipx install codex-telegram-bridge
```

윈도우 파워셸: `py --version` 과 `codex --version` 으로 먼저 확인하고, `pipx` 대신 필요한 방식으로 설치합니다(4.2 참고).

설치 확인:

```
codex-telegram-bridge --help
```

Codex 브릿지의 설치 확인 화면에는 `setup`, `doctor`, `uninstall`이 표시됩니다. 사용 절차는 공개 저장소의 최신 README를 정본으로 보세요.

6.3 [따라하기] 봇을 새로 팔까, 재활용할까

권장: Codex용 봇을 새로 하나 더 만드세요. 4장에서 만든 Claude 봇과 **별개 봇**을 쓰면, 폰에서 "이건 Claude, 저건 Codex" 하고 명확히 구분됩니다.

- BotFather에서 `/newbot` 다시 → 예: 이름 `AI Codex Demo`, 아이디 `aicodex_demo_2026_bot`.
- 새 토큰을 받고, 그 봇에 `/start`를 보내 chat_id를 확인합니다(2.3~2.4와 동일).

굳이 하나로 합칠 수도 있지만, 초보자는 **봇 2개(=조수 2명, 전화 2대)** 가 헷갈림이 적습니다.

6.4 [따라하기] 설정하고 첫 연결

공개 README의 기본 경로는 **보이는 Codex 세션을 먼저 켜고, 다른 터미널에서 마법사를 실행**하는 순서입니다.

터미널 1:

```
tmux -L codex new -s codex
codex
```

터미널 2:

```
codex-telegram-bridge setup
```

마법사는 토큰을 화면에 숨겨 입력받고, 텔레그램에서 `/start`를 기다려 chat_id를 감지한 뒤 `~/.config/telegram-agent-bridge.env`를 권한 600으로 만듭니다. 이어서 macOS의 launchd, 리눅스·WSL의 systemd, 윈도우의 사용자 시작 경로에 브릿지 서비스와 워치독을 설치합니다.

직접 설정 파일을 고칠 때는 다음처럼 구분하세요.

- `TAB_BOT_TOKEN`, `TAB_CHAT_ID`, `TAB_BRIDGE_MODE`, `TAB_AGENT`, `TAB_AGENT_CMD`, `TAB_STATE_DIR`, `TAB_WORKDIR` : 설치 방식과 실행 모드가 함께 쓰는 공통 설정.
- `CRB_TMUX_SOCKET`, `CRB_TMUX_SESSION`, `CRB_TMUX_SUBMIT_KEY` : 보이는 Codex TUI에 연결하는 `repl` 모드 전용 설정.
- `CRB_FLOW_MIRROR`, `CRB_REASONING_MIRROR` 등: 필요한 경우만 켜는 Codex REPL 옵션. `CRB_*`가 `TAB_*`의 대체 접두어라는 뜻은 아닙니다.

설치 뒤에는 다음 명령으로 설정·세션·서비스 상태를 확인합니다.

```
codex-telegram-bridge doctor
```

그다음 폰에서 Codex 봇에게 `/ping`을 보내고, 이어서 진짜 지시(`지금 폴더의 파일 목록 보여줘`)를 보내세요. `repl` 모드라면 질문이 터미널 1의 Codex 화면에도 나타나고 최종 답이 폰으로 돌아옵니다.



그림은 공개 브릿지의 응답 형식을 보여주는 예시입니다. 봇 이름과 chat_id는 독자 환경마다 다릅니다.

> 기대 결과
>
> Codex 봇 /ping에 생존 답이 오고, 이어서 파일 목록 지시의 최종 답이 옵니다. 설정 파일은 `~/.config/telegram-agent-bridge.env` 입니다.
>
> **다음 행동:** 6.5에서 봇 이름을 구분해 고정합니다. 그다음 7장입니다.

6.5 Claude 봇과 Codex 봇 안 헷갈리게

- **봇 이름**을 한눈에 갈리게 짓기: 이 책 예시처럼 `AI Bridge Demo` (Claude용) / `AI Codex Demo` (Codex용) 식으로, 한쪽에만 `Codex`를 넣어도 충분합니다.
- 텔레그램에서 두 봇 채팅을 **상단 고정(pin)** 해 두면 오가기 편합니다.
- 어느 봇에 말하고 있는지 항상 확인하고 지시하세요(엉뚱한 조수에게 시키면 엉뚱한 작업이 돌아갑니다).

7장. 재시작과 상시 구동 — 안 끊기게 만들기

여기까지 왔으면 연결은 됐습니다. 이제 **"자꾸 끊기지 않게"** 만들어, 밖에서 언제 말해도 답하도록 다집니다.

7.1 왜 자꾸 끊기나 — 세 가지 원인

1. **세션이 죽음:** 터미널 창을 닫거나 실수로 종료 → 안의 AI 세션도 꺼짐. → tmux로 해결(7.2).
2. **프로그램(브릿지)이 죽음:** 드물게 브릿지 자체가 멈춤. → 워치독으로 자동 복구(7.3).
3. **컴퓨터가 꺼짐/재부팅:** 정전·업데이트 등. → 부팅 시 자동 시작 등록(7.4).

컴퓨터 전원과 인터넷은 워치독으로 대신할 수 없습니다.

7.2 [따라하기] tmux로 세션 살려두기

3장에서 봤듯 tmux 안에서 돌리면 창을 닫아도 안 꺼집니다. 이미 돌고 있는 세션에 다시 **붙는(attach)** 법:

Claude:

```
tmux -L default attach -t claude
```

Codex (공개 README 정본):

```
tmux -L codex attach -t codex
```

보다가 빠져나올 때는 **`Ctrl+B` 를 누른 뒤 손 떼고 `D`**. 이걸 **디태치(detach)** 라고 하고, 세션은 계속 살아 있습니다.

윈도우(WSL)에서는 평범한 파워셸 창에서도 볼 수 있습니다.

```
WSL.exe -- tmux -L default attach -t claude
```

기대 결과

터미널을 닫았다가 다시 연 뒤에도 attach가 되고, 폰 /ping이 살아 있습니다.

다음 행동: 7.3에서 setup이 설치한 워치독을 doctor로 확인합니다.

7.3 [따라하기] 워치독으로 자동 복구

워치독(watchdog)은 브릿지가 살아 있는지 확인하고, 멈춰 있으면 사용자 서비스를 다시 켜는 지킴이입니다. 현재 공개 배포본은 Claude와 Codex 모두 설정 마법사가 서비스와 워치독을 함께 설치합니다. 따로 스크립트를 복사하는 대신 다음 공식 흐름을 사용하세요.

```
claude-telegram-bridge setup
claude-telegram-bridge doctor

codex-telegram-bridge setup
codex-telegram-bridge doctor
```

`doctor`가 서비스와 워치독 상태를 함께 보여 줍니다. 배포 패키지에는 `claude-telegram-bridge-watchdog`와 `codex-telegram-bridge-watchdog` 엔트리 포인트도 실제로 포함되지만, 초보자의 설치·복구 기준은 운영체제별 파일을 직접 손대는 대신 `setup`과 `doctor`입니다.

설치되는 감시 경로는 운영체제에 따라 다릅니다.

- **macOS:** 사용자 LaunchAgent가 주기적으로 상태를 확인합니다.
- **리눅스·WSL:** 사용자 systemd 서비스와 timer를 사용합니다.
- **윈도우:** 사용자 시작 경로와 예약 작업을 사용합니다.

컴퓨터 자체가 꺼져 있거나 인터넷이 끊긴 경우에는 워치독도 동작할 수 없습니다.

7.4 [따라하기] 컴퓨터 재부팅에도 살아남기

컴퓨터를 껐다 켜도 자동으로 다시 뜨게 하려면, **부팅 시 자동 시작(서비스 등록)** 이 필요합니다. 설정 마법사가 운영체제에 맞게 등록합니다(맥=launchd, 리눅스·WSL=systemd, 윈도우=사용자 시작 경로·예약 작업). 설치를 건너뛰지 않았다면 `doctor`에서 실제 등록 상태를 확인할 수 있습니다.

재부팅 후엔 **폰에서 `/ping` 한 번**으로 살아났는지 확인하는 습관을 들이세요. `/ping`만 확인하고 끝내지 말고, 5장처럼 실제 지시 한 줄도 이어서 보냅니다.

7.5 잘 돌고 있나 확인하는 습관 (doctor)

가장 든든한 한 줄. 토큰·tmux·훅·서비스 상태를 한꺼번에 점검합니다.

```
claude-telegram-bridge doctor
```

어디가 막혔는지 항목별로 알려 줘서, 문제 생겼을 때 제일 먼저 칠 명령입니다. Codex는 `codex-telegram-bridge doctor` 입니다.

8장. 실패별 원인·확인·다음 행동

증상만 읽지 말고, **원인** → **확인 명령** → **다음 행동** 순으로 따라 가세요. 한 번에 여러 처방을 섞지 않습니다.

8.1 /ping은 오는데 진짜 답이 안 와요

원인	SessionStart 훅이 없거나, 등록 후 Claude 세션을 재시작하지 않음
확인	`claude-telegram-bridge doctor` 의 hook 줄, tmux 세션에 질문이 보이는지
다음 행동	4.5 훅 등록 → Claude 세션에서 `/clear` 한 번. 브릿지 터미널 로그가 “무엇을 기다리는 중인지” 알려 줍니다

8.2 봇이 아무 반응이 없어요

원인	토큰 불일치, chat_id 불일치, 인터넷 끊김, 브릿지 미실행
확인	`claude-telegram-bridge doctor`
다음 행동	① 토큰 ② chat_id ③ 네트워크 ④ 브릿지 실행 순서로 하나만 고칩니다

8.3 externally-managed-environment 오류가 떠요

원인	우분투/WSL에서 `pip install` 을 시스템 파이썬에 직접 실행
확인	오류 문구에 `externally-managed-environment`
다음 행동	**pipx로 설치**(4.2). 정 직접 pip를 쓰려면 명령 끝에 `--break-system-packages` 를 붙이되, 스스로 관리한다는 뜻임을 이해하고 쓰세요

8.4 윈도우에서 python3가 없대요

원인	파워셸의 기본 실행 이름이 `py` 인 경우가 많음. 고장이 아님
확인	`py --version`
다음 행동	`py -m pip install --user pipx` . 첫 연결 실습은 WSL 안에서 `python3` 로 하는 경로를 권합니다. Codex는 `py --version` · `codex --version` 으로 먼저 확인

8.5 답이 두 번 와요 / 메시지가 꼬여요

원인	텔레그램 답장 도구를 따로 켜 중복 발송. 세션이 두 개
확인	봇이 하나인지, tmux 세션이 하나인지, 다른 브릿지가 떠 있는지
다음 행동	브릿지 재시작 후 `doctor` . 브릿지에는 중복 발송 가드가 포함돼 있습니다

8.6 그 외 자주 막히는 지점

증상	원인	확인	다음 행동
봇에게 토큰을 보내 버렸다	비밀값을 채팅에 붙여 넣음	BotFather 대화	즉시 재발급(부록 A.2)
창을 닫았더니 꺼졌다	tmux 밖에서 실행	`tmux ls`	7.2 attach / 새로 시작
재부팅 후 안 된다	서비스 미등록 또는 절전	`doctor` , `/ping`	7.4 후 5장 양방향 확인을 다시
Codex /ping만 되고 답이 없다	터미널 1의 Codex 세션이 없음	`tmux -L codex attach -t codex`	6.4 순서대로 세션을 먼저 켬

9장. 학습용 가상 사례 — 첫 연결부터 복구까지

아래 인물·일정·지시는 **학습용 가상 사례**입니다. 실제 매출, 절감 시간, 성과를 보여 주는 기록이 아닙니다.

9.1 사례 한 줄 요약

가상 독자 **민서**는 집에 macOS 컴퓨터가 있고, 이미 Claude Code에 로그인돼 있습니다. 목표: 카페에서 폰으로 "지금 폴더 알려줘"를 보내고, 최종 답을 받는 것. 그다음 컴퓨터를 재시작한 뒤에도 `/ping`과 실제 지시가 다시 되는지 확인합니다.

9.2 [따라하기] 민서의 당일 순서

이 순서는 1~8장의 명령을 한 줄로 이은 것입니다. 새 패키지 절차가 아닙니다.

1. **적합성.** `claude --version`, `tmux -V`, `python3 --version`. 컴퓨터는 켜 둔다.
2. **토큰.** BotFather `/newbot` → 토큰은 채팅에 안 붙인다.
3. **연결.** `pipx install claude-telegram-bridge` → `claude-telegram-bridge setup`.
4. **양방향.** 폰에서 `/ping` 다음 `지금 어느 폴더에서 돌고 있어?`.
5. **재시작.** 컴퓨터를 재시작한 뒤 `claude-telegram-bridge doctor`, 다시 `/ping`과 실제 지시.

민서가 카페에서 보내는 문장 예:

```
어제 만든 메모 파일 이름만 나열해 줘. 내용은 바꾸지 마.
```

기대 결과: 파일 이름 목록이 폰으로 돌아온다. 기대한 목록이 아니면 8.1을 본다. 이 문장으로 매출이 난다는 주장은 하지 않는다.

9.3 Windows WSL 경로만 다른 가상 사례

학습용 가상 사례. 가상 독자 **지훈**은 Windows에 WSL이 있습니다. 파워셸에서 `python3`가 없어도 중단하지 않습니다. WSL을 연 뒤 3.2와 4.2를 **Linux 명령으로** 따라 갑니다. 파워셸의 `py -m pip install --user pipx`는 pipx를 까는 보조일 뿐, 첫 `/ping` 실습의 정본은 WSL입니다.

9.4 연결한 뒤에 쓸 수 있는 기능 10가지

4~7장을 끝낸 뒤에만 시도하세요. 각 줄은 "상황 → 폰에서 보낼 말 → 벌어지는 일"입니다. 켠 기능을 README에서 확인하지 않은 채 고장으로 단정하지 마세요.

1. **짧은 수정 지시** — `어제 로그인 버튼 색을 파란색으로 바꿔줘` → 집 컴퓨터가 파일을 고치고 최종 답을 보냄.
2. **사진 보내 분석** — 에러 화면을 찍어 봇에 전송 → AI가 그 이미지를 보고 원인·해결을 답함. *(사진은 컴퓨터에 저장된 뒤 분석에 쓰입니다.)*
3. **긴 작업 진행 보기** — 흐름 카드(flow mirror)는 기본 꺼짐입니다. 켜려면 각 README의 `CLB_FLOW_MIRROR=1` / `CRB_FLOW_MIRROR=1`.
4. **상태 한눈에 (`/context`)** — 봇에 `/context` → 현재 상태 화면을 **그림으로** 받아봅니다.
5. **모델 바꾸기 (`/model`)** — `/model` → 버튼으로 모델을 골라 바로 적용(대화형 화면이 폰을 멈추지 않게 버튼으로 처리).
6. **음성으로 지시** — 음성 메모를 보내면(전사 기능을 켠 경우) 말한 내용이 지시로 들어갑니다.
7. **파일·문서 넘기기** — 텔레그램으로 파일을 보내면 컴퓨터에 저장되어 작업 대상이 됩니다.
8. **여러 프로젝트 오가기** — Claude 봇/Codex 봇을 나눠 쓰거나, 세션을 바꿔 가며 프로젝트를 오갑니다.
9. **추천답변 톡톡** — 답 아래 추천 문구. Claude 다리는 이 버블이 **기본 꺼짐**입니다. 안 뜨면 고장이 아니라 설정입니다.
10. **메모·초안** — 컴퓨터 앞이 아니어도 되는 정리·번역·초안을 한 건씩 옮겨 봅니다.

밤사이 자동으로 굴리기는 편리하지만, AI가 사람 확인 없이 작업하게 되는 만큼 **비용·안전·약관**을 스스로 판단하고 켜세요(1.5·부록 A).

9.5 사례가 끝난 뒤 독자가 남기는 기록

빈칸은 이 책에 제출하지 않습니다. 자기 컴퓨터에만 적으세요. 토큰은 적지 않습니다.

확인	내 결과	다음 행동
적합성 점검	(통과/보류)	보류면 1.4에서 멈춤
`/ping`	(옴/안 옴)	안 오면 8.2
실제 지시	(옴/안 옴)	안 오면 8.1
재시작 후 `/ping`	(옴/안 옴)	안 오면 7.4
운영체제 경로	macOS / Linux / WSL	1.3과 맞는지

부록 A. 보안·토큰 관리 상식

A.1 토큰이 노출되면 벌어지는 일

봇 토큰은 **봇의 열쇠**입니다. 새면 남이 당신 봇으로 메시지를 주고받을 수 있습니다. 그래서 토큰은 **로컬 파일에만**, 권한을 잠가(600) 보관하고, 채팅·스크린샷·공개 저장소에 절대 올리지 않습니다.

A.2 [따라하기] 토큰 안전 보관·교체

- 보관: `token.json`을 권한 600으로(4.4). 화면에 안 남게 Python `getpass` 입력을 사용하기.
- **노출됐다면 교체:** BotFather에서 해당 봇 → `/revoke`(또는 재발급) → **새 토큰**을 받아 `token.json`을 갈아끼우고 브릿지 재시작. 옛 토큰은 즉시 무효가 됩니다.

A.3 `--dangerously-skip-permissions`, 켤까 말까

이 옵션은 Claude가 **매번 "해도 돼요?"를 묻지 않고** 작업하게 합니다. 밖에서 폰으로 부릴 땐 확인 버튼을 눌러 줄 사람이 없어서 편리하지만, 그만큼 **AI가 물어보지 않고 당신 컴퓨터에서 움직인다**는 뜻입니다.

- **켠다:** 원격 자동화의 편의 ↑. 대신 위험한 작업도 확인 없이 실행될 수 있음을 이해하고, 중요한 데이터가 있는 환경에선 신중히.
- **끈다**(그냥 `claude`): 안전하지만, 확인 창이 뜨면 그 폰 지시는 당신이 터미널에서 답할 때까지 멈춤.

정답은 없습니다. **당신의 데이터 민감도와 편의 사이에서** 고르세요.

부록 B. 명령·환경변수 치트시트

이 부록만으로도 설치 후 확인·복구에 필요한 명령을 다시 찾을 수 있습니다. 외부 페이지가 없어도 여기 있는 명령을 복사해 쓰면 됩니다.

B.1 자주 쓰는 명령

```
# Claude 브릿지
claude-telegram-bridge setup       # 설치 마법사
claude-telegram-bridge doctor      # 설정·훅·서비스·워치독 점검
claude-telegram-bridge uninstall   # 제거
tmux -L default attach -t claude   # Claude 세션 다시 보기

# Codex 브릿지
codex-telegram-bridge setup        # 설치 마법사
codex-telegram-bridge doctor       # 설정·세션·서비스·워치독 점검
codex-telegram-bridge uninstall    # 제거
tmux -L codex attach -t codex      # Codex 세션 다시 보기
```

tmux에서 빠져나올 때는 `Ctrl+B`를 누른 뒤 `D`를 누릅니다. AI 세션은 계속 실행됩니다.

B.2 Codex 설정 파일과 핵심 환경변수

설정 마법사가 만드는 기본 파일은 `~/.config/telegram-agent-bridge.env`입니다.

```
# 공통 설정 — setup이 작성
TAB_BOT_TOKEN       # BotFather 토큰(비밀)
TAB_CHAT_ID         # 허용할 내 텔레그램 채팅 하나
TAB_BRIDGE_MODE     # repl 또는 exec
TAB_AGENT           # codex
TAB_AGENT_CMD       # codex 실행 명령
TAB_STATE_DIR       # 폴링·대화 상태 저장 위치
TAB_WORKDIR         # Codex 작업 폴더

# 보이는 TUI를 쓰는 repl 모드 전용
CRB_TMUX_SOCKET     # 기본 codex
CRB_TMUX_SESSION    # 기본 codex
CRB_TMUX_SUBMIT_KEY # 기본 Tab

# 선택 기능
CRB_FLOW_MIRROR
CRB_REASONING_MIRROR
CRB_AUDIO_TRANSCRIBE_CMD
```

`TAB_*`는 공통 설정, `CRB_*`는 Codex REPL의 연결·선택 기능입니다. 자세한 전체 목록과 기본값은 공개 저장소의 `config.example.env`를 보세요. Claude 브릿지는 별도 프로그램이며 `~/.config/claude-telegram-bridge/bridge.env`의 `CLB_*`를 사용합니다.

닫는 글 — 이용 범위와 다음 행동

컴퓨터가 켜져 있고 인터넷이 되는 동안 폰 한 통이면 집 AI에게 일을 시킬 수 있습니다. 이 책은 필자 봇을 빌려 드리거나 수익을 보장하지 않습니다.

다 읽은 뒤 바로 할 일:

1. 1.4 점검표를 다시 본다.
2. 5장 양방향 확인을 자기 봇으로 한 바퀴 돈다.
3. 재시작 후 `/ping`과 실제 지시 한 줄을 다시 보낸다.
4. 막히면 8장 표의 “다음 행동”만 실행한다.

더 깊이 들어가고 싶다면 각 도구의 공식 저장소 README를 보세요. 거기가 살아 있는 정본입니다. 이 책이 영원히 화면을 같다고 약속하지는 않습니다.

구매자 본인의 학습·업무용으로 이 PDF를 읽고 인쇄하고 명령을 복사할 수 있습니다. 무단 복제·재배포·재판매는 금지합니다. 문의는 구매하신 판매 페이지의 메시지 기능을 이용하세요.

(끝)